
A note on Monte Carlo Tree Search

Sharanya Ranka

Department of Computer Science
University of California, San Diego
La Jolla, CA
smranka@ucsd.edu

1 Rough Work

A quick summary on how this works

A deep dive

Philosophies Computer Science - Formulating problems as search problems AI - Adding Heuristic search methods

Ponderings Positive Bias (Upper Confidence Bound) Generating solutions as a search rather than "creation" (Being in better situations) Getting out of Local optima?

Monte Carlo vs Las Vegas (the usefulness of stopping the search in the middle and getting an approximate answer)

Any further methods (combines MCTS with Neural networks, Reinforcement learning) The large complexity of Go. Simple MCTS does not help (sparse rewards)

Aid in the form of Value and Policy functions - Recognizing patterns - Use data/samples effectively

2 Quick Summary

The AI Agent you're playing against uses Monte Carlo Tree Search (MCTS) with the Upper Confidence Bound for Trees (UCT) *heuristic* in order to (relatively) quickly come up with responses for moves. The entire evolution of a game of Connect4 can be described in the form of a *GameTree*. Each game starts at the root node/state (an empty board) and proceeds along edges (moves by either player) to reach next states (a board with an additional yellow/red coin placed compared to the previous state).

While it is possible to simply brute-force search all the way to the end of the game, to determine what is the best move to play, such techniques tend to be extremely computationally intensive. A weak upper bound on the number of nodes would be of the order of $O(m^d)$ where m is the number of move options available per turn (< 7) and d is the total number of moves in the game ($d < 6 \cdot 7$)

3 A Deeper Dive

Monte Carlo Tree Search (MCTS) attempts to estimate the "goodness" of next moves. It does so by playing a large number of games from the current position, and computing the average number of times the current player won after choosing the particular action. $Q(s, a)$ is the estimated goodness of taking action a from state s . Instead of playing completely (uniformly) randomly, we can use experience gathered from previous playthroughs to *inform* our search. This helps us divert more computation towards exploring actions that seem promising. This technique is called bootstrapping (computing estimates on the basis of other estimates). The Upper Confidence Bound for Trees (UCT) heuristic aims to dynamically control exploration of alternate actions by providing an optimistic estimate of how good an action could be based on current experience. In essence, since we're only

estimating the goodness based on samples, it may be different from the "true" goodness of action a . Given that exploration of an action (which also leads to more accurate estimates) depends on how good the experience till now was, it is useful to *positively bias* these estimates so as to encourage exploration.

The MCTS algorithm consists of 4 stages for each play-through of a game: Selection, Expansion, Simulation and Backpropagation.

3.1 Selection

The selection stage starts at the current state, and repeatedly *selects* the next action to take until it reaches a leaf state

3.2 Expansion

The expansion step is a quick one, where you simply add the children of the current (leaf) state to the GameTree, and the current state stops being a leaf.

3.3 Simulation

At this point, we are still at the (previously leaf) state. Since we do not have any understanding of whether we are at a state that favours the yellow or red token, we can continue randomly selecting moves until a final game state is reached (win for yellow/red or a draw). If we are at a state that highly favours yellow, there is a high probability that we reach a state where yellow has one, if the state is draw-ish, this would too reflect as a high probability of draw verdicts from this state. This stage ends when we reach a final game state - a state where either red/yellow is the winner or its a draw.

3.4 Backpropagation

The result of the play-through needs to be communicated to all previous states so that the "experience" can be updated and used for subsequent moves. This is done by retracing our steps back up the tree to the current game-state. For example, if the yellow token won, the actions taken by yellow would register that as a positive experience and the actions taken by red would register it as negative experience

4 AI Inspired Philosophy!

5 Going Beyond MCTS

Systems like AlphaZero also use Monte Carlo Tree Search to inform their search through the GameTree. However, games like Chess and Go have much larger depth (number of moves in a game) and more importantly breadth/branching factor (number of options per move). The GameTree becomes very deep and broad, and even previous heuristics (like UCT) fail to search for good moves in a reasonable amount of time.

Pairing the Monte Carlo Tree Search with Neural Networks, however, effectively solves the previous problem. The Neural Network serves to provide value estimates for next states in the game (i.e. how good each next-move is). This serves as an initial *bias* to the Monte Carlo Tree Search which further improves these estimates by playing the games till the end. The "improved" estimates for goodness of moves is used to train the next iteration of the Neural Network, which then informs the next round of self-play... In this way, even if the Neural Network starts with rather trivial "uniform" predictions, it improves with each round of play. The Neural Network employed is a Convolutional Neural Network (CNN) which has some useful biases (local field of reception, shared weights in the filters) compared to fully-connected Neural Networks, that massively reduce the amount of computation required to achieve similar performance. These have proved useful in a wide range of Vision based tasks too.